

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе
по дисциплине «Естественно языковой интерфейс в интеллектуальных
системах»
на тему:

**Разработка информационно-поисковой системы.
Реализация методов оценки качества ее работы**

Студенты гр. 321701

И. А. Политыко

В. А. Лукашов

Руководитель

Ю. Б. Крапивин

Минск 2026

Содержание

Перечень условных обозначений	3
1 Алгоритмы информационно-поисковой системы	5
1.1 Предобработка текста	5
1.2 Инвертированный индекс	5
1.3 Вероятностная модель ранжирования	5
1.4 Псевдорелевантная обратная связь	6
1.5 Отсечение нерелевантного хвоста	6
1.6 Базовая векторная модель	7
1.7 Метрики качества поиска	7
2 Разработка информационно-поисковой системы	8
2.1 Выбор средств реализации	8
2.2 Описание используемых готовых компонентов	9
2.3 Структура разработанной системы	9
2.4 Структура базы данных системы	13
2.5 Основные алгоритмы реализации компонентов	14
2.6 Информация о тестовой коллекции документов	18
2.7 Результаты тестирования системы	20
2.8 Результаты оценки по метрикам	20
2.9 Анализ результатов и предложения по улучшению	24
2.10 Вывод по разделу	25
Заключение	26
Список использованных источников	27

Перечень условных обозначений

API — Application Programming Interface, программный интерфейс приложения;

REST — Representational State Transfer, архитектурный стиль взаимодействия компонентов;

JSON — JavaScript Object Notation, текстовый формат обмена данными;

HTTP — HyperText Transfer Protocol, протокол передачи гипертекста;

SQL — Structured Query Language, язык структурированных запросов;

СУБД — система управления базами данных;

БД — база данных;

ЛВС — локальная вычислительная сеть;

ИПС — информационно-поисковая система;

СИП — система информационного поиска;

ЕЯ — естественный язык;

ПОД — поисковый образ документа;

ПОЗ — поисковый образ запроса;

TF — Term Frequency, частота термина в документе;

IDF — Inverse Document Frequency, инверсная документная частота;

TF-IDF — Term Frequency — Inverse Document Frequency, статистическая мера значимости термина;

BM25 — Best Match 25, вероятностная модель ранжирования Okapi;

BIM — Binary Independence Model, бинарная модель независимости терминов;

RSJ — Robertson–Sparck Jones, вес Робертсона – Спарк Джонс;

RSV — Retrieval Status Value, статус релевантности документа;

PRF — Pseudo-Relevance Feedback, псевдорелевантная обратная связь;

MAP — Mean Average Precision, средняя средняя точность;

nDCG — Normalized Discounted Cumulative Gain, нормализованный дисконтированный накопленный выигрыш;

P@k — Precision at k, точность на первых k результатах;

F-мера — мера качества поиска, объединяющая точность и полноту;

NLP — Natural Language Processing, обработка естественного языка;

SPA — Single Page Application, одностраничное веб-приложение;

CSV — Comma-Separated Values, формат с разделителями-запятыми;

PDF — Portable Document Format, переносимый формат документов;

DOCX — формат документов Microsoft Word;
RTF — Rich Text Format, формат расширенного текста;
HTML — HyperText Markup Language, язык гипертекстовой
разметки.

1 Алгоритмы информационно-поисковой системы

Разработанная система реализует вероятностную стратегию поиска по варианту 5 задания: документы ранжируются по убыванию оценки вероятности релевантности запросу. Пользователь формулирует запрос на естественном русском языке; система автоматически строит поисковый образ запроса, сопоставляет его с инвертированным индексом коллекции и возвращает упорядоченный список документов.

1.1 Предобработка текста

Документы и запросы проходят одинаковый конвейер преобразований, чтобы поисковые образы имели одинаковую структуру:

- а) приведение к нижнему регистру и замена буквы «ё» на «е»;
- б) токенизация регулярным выражением, допускающим внутренние дефисы;
- в) лемматизация русскоязычных словоформ анализатором `ru morphology3`;
- г) фильтрация стоп-слов и токенов короче двух символов.

Латинские термины и аббревиатуры (SQL, IP, Ethernet) сохраняются в исходном написании: морфологический словарь русского языка к ним неприменим, а сами строки уже являются нормальной формой. Результат предобработки — список лемм, образующих поисковый образ документа или запроса.

1.2 Инвертированный индекс

Для каждого документа строится инвертированный индекс: термин указывает на список документов, в которых он встречается, с указанием частоты. Дополнительно хранятся документная частота термина n_i и длина документа $|d|$ в леммах. Эти статистики достаточны для вычисления весов Робертсона – Спарк Джонс и оценки BM25 без повторного чтения текстов коллекции.

1.3 Вероятностная модель ранжирования

В основе лежит принцип вероятностного ранжирования: оптимальная выдача упорядочивает документы по убыванию вероятности их релевантности запросу. На практике вычисляется монотонное преобразование этой вероятности — статус релевантности RSV.

Вес термина без обратной связи задаётся сглаженной формулой Робертсона – Спарк Джонс:

$$w_i = \log \left(1 + \frac{N - n_i + 0,5}{n_i + 0,5} \right), \quad (1.1)$$

где N — число документов в коллекции; n_i — число документов, содержащих термин i . Слагаемое 1 внутри логарифма устраняет отрицательные веса для терминов, встречающихся более чем в половине документов, сохраняя монотонность: редкий термин получает больший вес, чем частый.

Ранжирование выполняется по формуле Окари BM25:

$$\text{RSV}(d, q) = \sum_{i \in q} w_i \cdot \frac{f_{i,d} (k_1 + 1)}{f_{i,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}, \quad (1.2)$$

где $f_{i,d}$ — частота термина i в документе d ; $|d|$ — длина документа; avgdl — средняя длина документа коллекции. Параметр $k_1 = 1,2$ задаёт насыщение частоты: вклад термина растёт быстро при первых вхождениях и выходит на плато. Параметр $b = 0,75$ нормирует оценку по длине документа, не давая преимущество длинным текстам только за счёт объёма.

1.4 Псевдорелевантная обратная связь

Полная вероятностная модель предполагает известное множество релевантных документов. Пользователь его не задаёт, поэтому применяется псевдорелевантная обратная связь. После первого прохода BM25 верхние $R = 5$ документов объявляются условно релевантными, и вес термина пересчитывается:

$$w_i = \log \frac{(r_i + 0,5) / (R - r_i + 0,5)}{(n_i - r_i + 0,5) / (N - n_i - R + r_i + 0,5)}, \quad (1.3)$$

где R — размер псевдорелевантного множества; r_i — число документов этого множества, содержащих термин i . Второй проход BM25 выполняется уже с уточнёнными весами. В интерфейсе обратная связь включается отдельным флажком: на малой разнородной коллекции она может понижать точность, подмешивая общие термины из соседних документов.

1.5 Отсечение нерелевантного хвоста

Оценка BM25 положительна для любого документа, содержащего хотя бы один термин запроса. Чтобы выдача не засорялась слабо связанными текстами, документы, чья оценка ниже 0,25 от оценки лидера, отбрасываются. Порог подобран экспериментально: он сохраняет тематически близкие результаты и отсекает случайные совпадения.

1.6 Базовая векторная модель

Для сравнения реализована векторная модель с логарифмическим TF-IDF и косинусной близостью:

$$\text{sim}(d, q) = \frac{(\vec{D}, \vec{Q})}{\|\vec{D}\| \cdot \|\vec{Q}\|}, \quad w_i^d = (1 + \log f_{i,d}) \cdot (\log(N/n_i) + 1). \quad (1.4)$$

Эта модель не используется как основная: она соответствует другому варианту задания и служит только контрольной точкой при оценке качества.

1.7 Метрики качества поиска

Пусть Rel — множество эталонных релевантных документов по запросу, R — множество документов, выданных системой. Тогда

$$P = \frac{|R \cap Rel|}{|R|}, \quad \text{Recall} = \frac{|R \cap Rel|}{|Rel|}, \quad F_\beta = \frac{(1 + \beta^2) P \cdot \text{Recall}}{\beta^2 P + \text{Recall}}. \quad (1.5)$$

Дополнительно вычисляются точность на глубине k ($P@5$, $P@10$), R-Precision, средняя точность AP, усреднённая по запросам MAP, нормализованный дисконтированный выигрыш nDCG@10 и 11-точечная интерполированная кривая полноты-точности. Параметр β задаёт акцент: $F_{0,5}$ сильнее штрафует падение точности, F_2 — падение полноты, F_1 уравнивает оба свойства.

2 Разработка информационно-поисковой системы

2.1 Выбор средств реализации

Система реализована по клиент-серверной схеме и разворачивается в локальной вычислительной сети средствами Docker Compose. При разработке использован следующий технологический стек.

- **Python 3.11** — язык серверной части: предобработка текста, вероятностная модель, REST API, оценка качества;
- **FastAPI** — веб-фреймворк; маршрутизация HTTP-запросов, валидация входных данных моделями Pydantic, автоматическая документация OpenAPI;
- **Uvicorn** — ASGI-сервер для запуска FastAPI;
- **PostgreSQL 16** — реляционная СУБД; хранит документы, инвертированный индекс, эталонную разметку и журналы;
- **psycopg2** — драйвер PostgreSQL для Python;
- **pymorphy3** — морфологический анализатор русского языка; лемматизация словоформ;
- **matplotlib** — построение графиков метрик качества;
- **pypdf**, **python-docx** — извлечение текста из PDF- и DOCX-файлов;
- **React 18**, **Vite**, **react-router-dom** — одностраничный веб-интерфейс;
- **nginx** — раздача собранного фронтенда и проксирование запросов /api на бэкенд;
- **Docker Compose** — контейнеризация трёх сервисов и воспроизводимое развёртывание в ЛВС.

Вероятностная модель выбрана в соответствии с вариантом задания. Расчёт RSV по формуле (1.2) выполняется SQL-запросом на стороне СУБД: индекс уже лежит в таблицах **terms** и **postings**, и нет необходимости выгружать коллекцию в память приложения.

FastAPI выбран как лёгкий фреймворк с явной схемой запросов: тело поискового запроса, параметры модели и флажок обратной связи проверяются до входа в бизнес-логику. React обеспечивает интерактивный интерфейс — элемент искусственного интеллекта по варианту 5: автодополнение, исправление опечаток и подсветка найденных слов не требуют перезагрузки страницы.

Контейнеризация закрывает требование о сфере применения «локальная вычислительная сеть»: сервис публикуется на порту 8080 и доступен всем узлам сети по адресу узла, на котором запущен Docker.

2.2 Описание используемых готовых компонентов

– **pymorphy3** — для каждого русскоязычного токена выбирается наиболее вероятная нормальная форма. Словарь **pymorphy3-dicts-ru** встраивается в Docker-образ, сетевые обращения при работе не требуются;

– **FastAPI / Uvicorn** — REST API и раздача описания эндпоинтов. CORS открыт, поскольку фронтенд и бэкенд в разработке могут работать на разных портах;

– **PostgreSQL** — расчёт BM25 выполняется одним запросом с **unnest** массивов идентификаторов и весов терминов. Агрегат **array_agg** возвращает список совпавших лемм для подсветки;

– **matplotlib** — бэкенд Agg строит PNG без графической подсистемы. Для интерфейса графики кодируются в base64, для отчёта сохраняются в каталог **png/**;

– **pypdf / python-docx** — подключаются только при загрузке соответствующего формата;

– **React** — клиентские страницы поиска, документов, загрузки, оценки качества и справки. Взаимодействие с сервером — **fetch** по JSON API;

– **nginx** — **try_files** отдаёт SPA, **proxy_pass** направляет **/api/** на сервис backend;

– **Docker Compose** — оркестрация сервисов db, backend и frontend с проверкой готовности базы данных.

2.3 Структура разработанной системы

Система состоит из трёх уровней: клиентского (React SPA за nginx), серверного (FastAPI) и уровня данных (PostgreSQL). Общая схема приведена на рисунке 2.1.

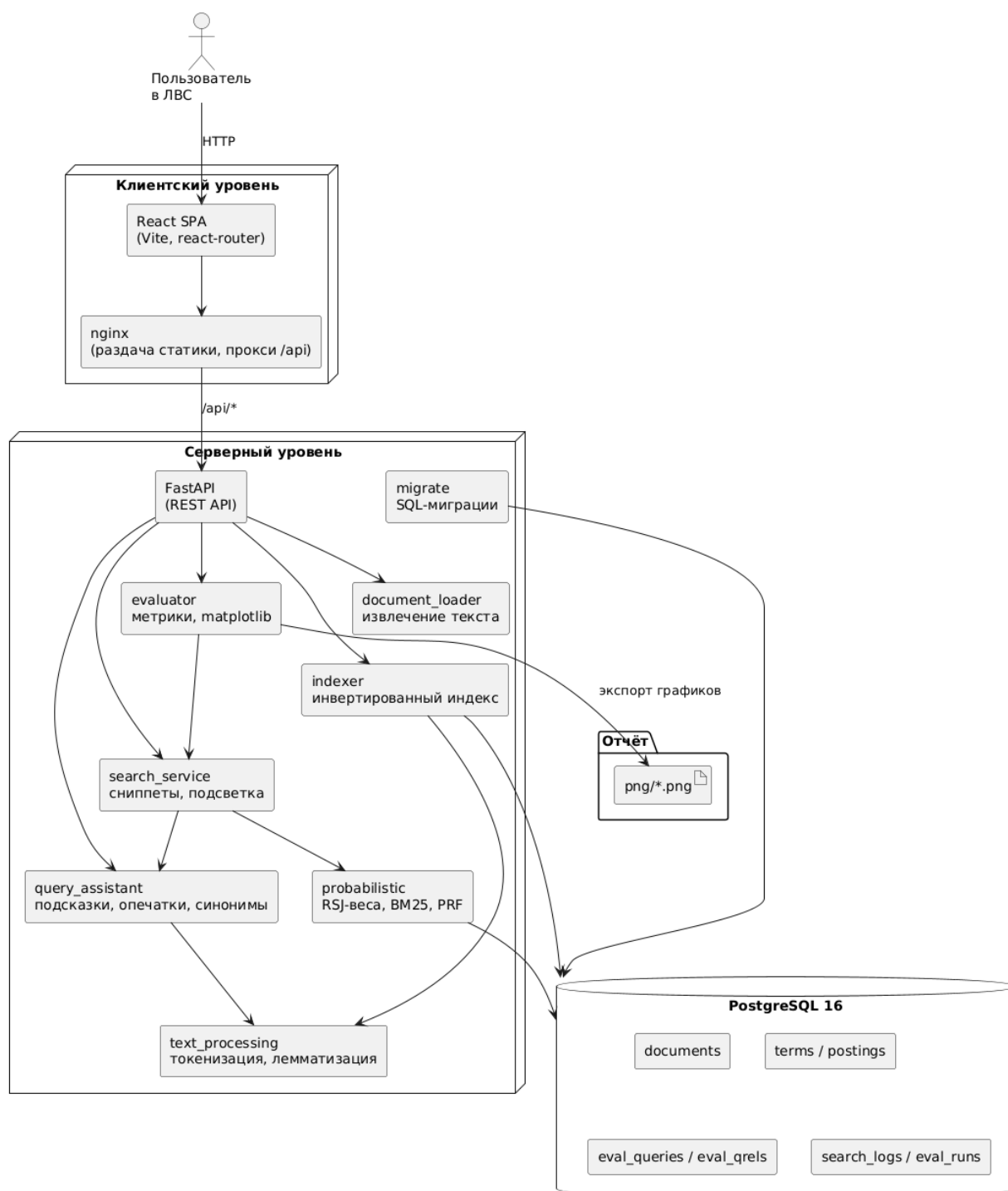


Рисунок 2.1 – Структура информационно-поисковой системы.

- Серверная часть разделена на модули по ответственности:
- `main.py` — точка входа, REST-эндпоинты и жизненный цикл приложения (миграции, загрузка корпуса);
 - `text_processing.py` — токенизация, лемматизация, фильтрация стоп-слов;
 - `indexer.py` — построение и обновление инвертированного индекса;
 - `document_loader.py` — извлечение текста из файлов разных форматов;

матов;

- `probabilistic.py` — веса RSJ, BM25, псевдорелевантная обратная связь, baseline TF-IDF;
- `query_assistant.py` — автодополнение, исправление опечаток, расширение синонимами, сниппеты;
- `search_service.py` — оркестрация поиска, отсечение хвоста, журналирование;
- `evaluator.py` — метрики качества и графики;
- `migrate.py` — применение SQL-миграций;
- `bootstrap.py` — первичная загрузка коллекции и эталонной разметки.

Клиентский уровень содержит пять страниц: поиск, список документов, загрузка, оценка качества и справка. Основной сценарий — обработка поискового запроса — показан на рисунке 2.2. Внешний вид интерфейса приведён на рисунках 2.3–2.5.

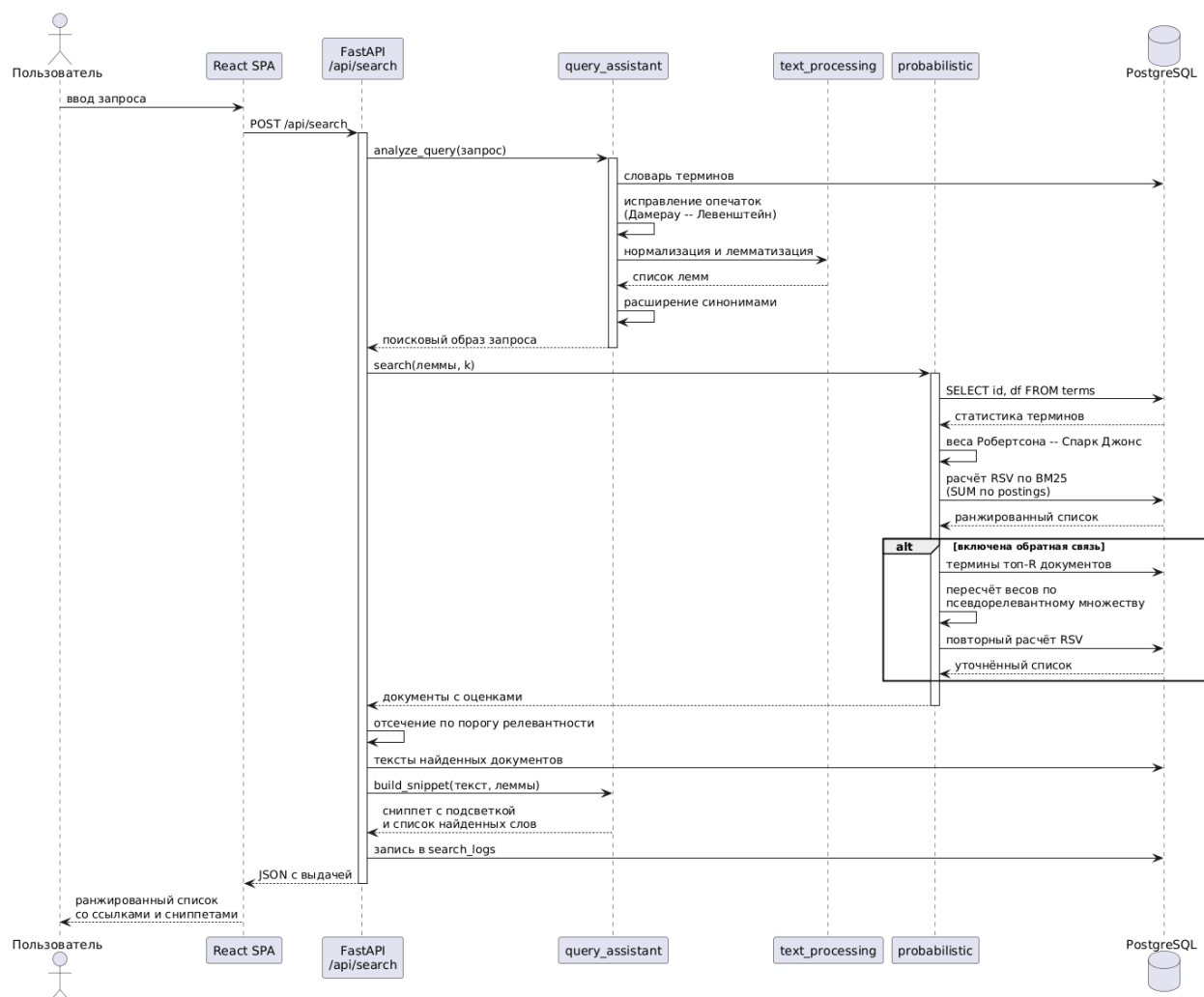


Рисунок 2.2 – Диаграмма последовательности обработки поискового запроса.

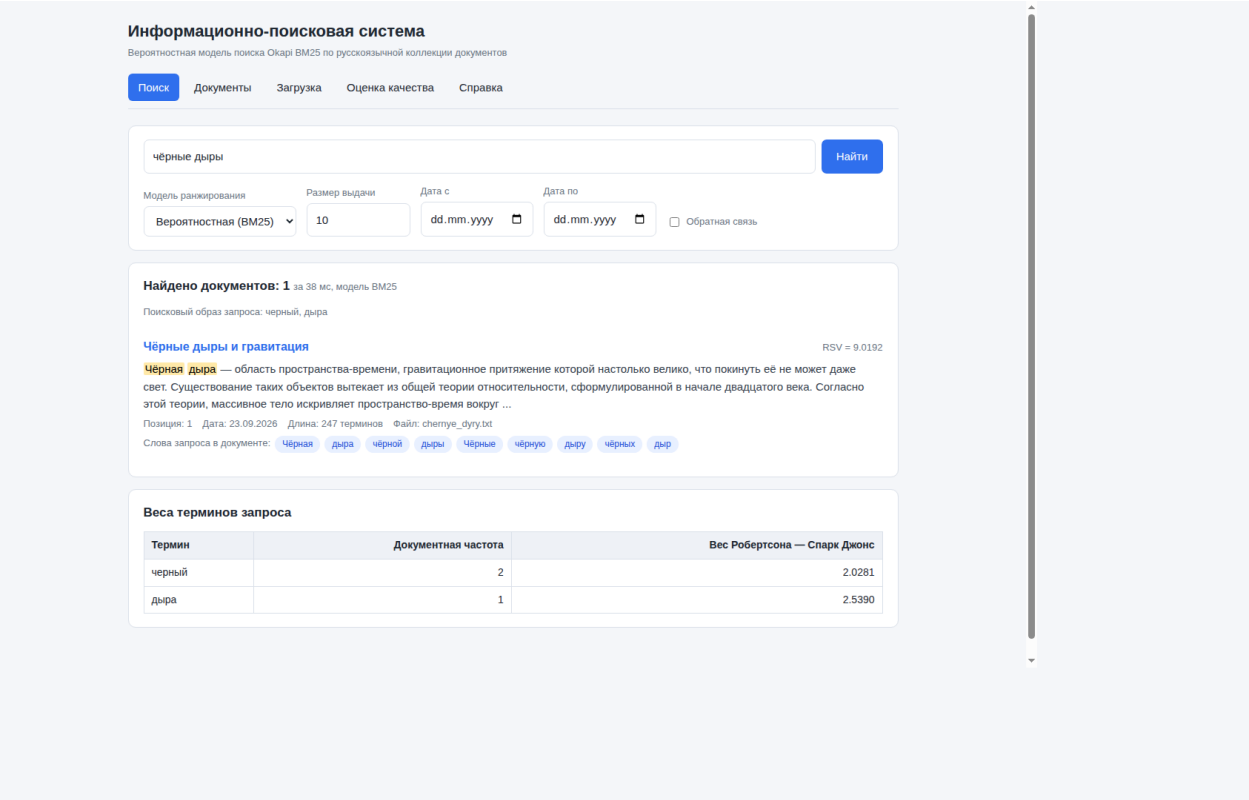


Рисунок 2.3 – Страница поиска с результатами запроса.

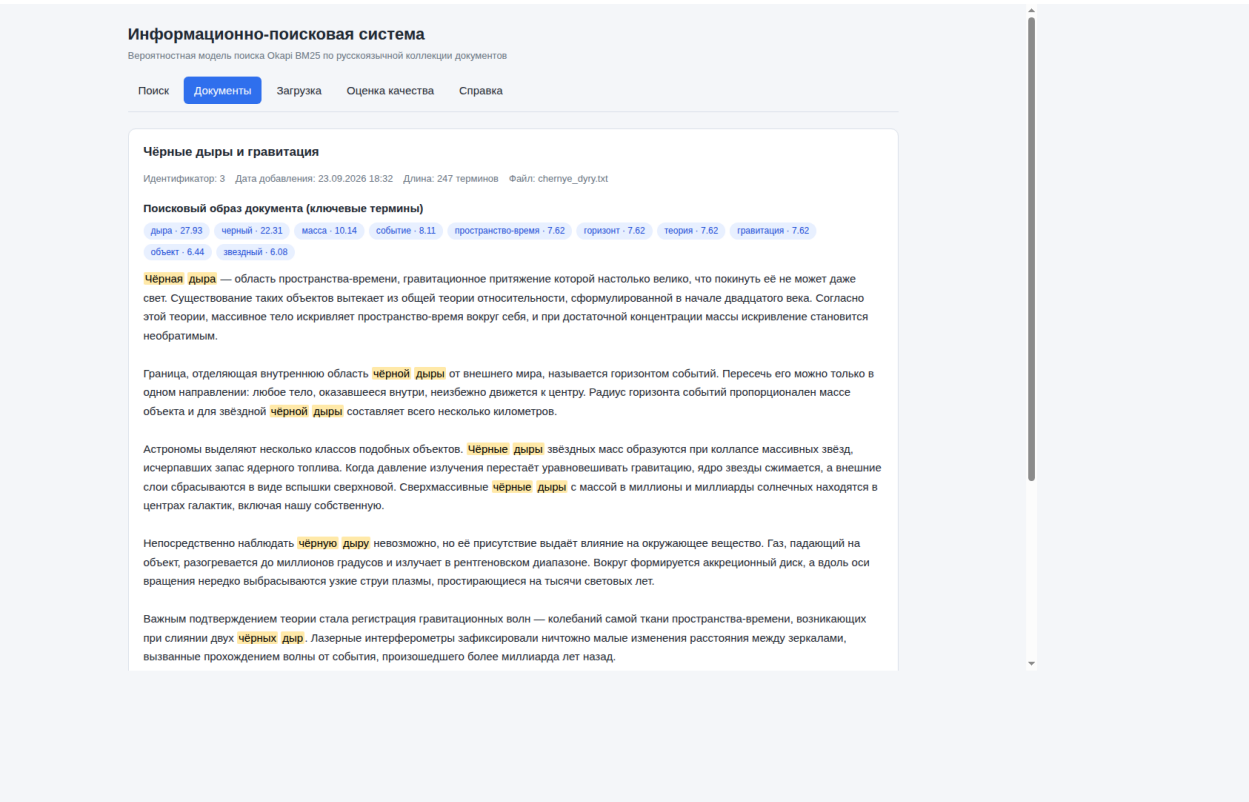


Рисунок 2.4 – Карточка документа с текстом и сниппетом.

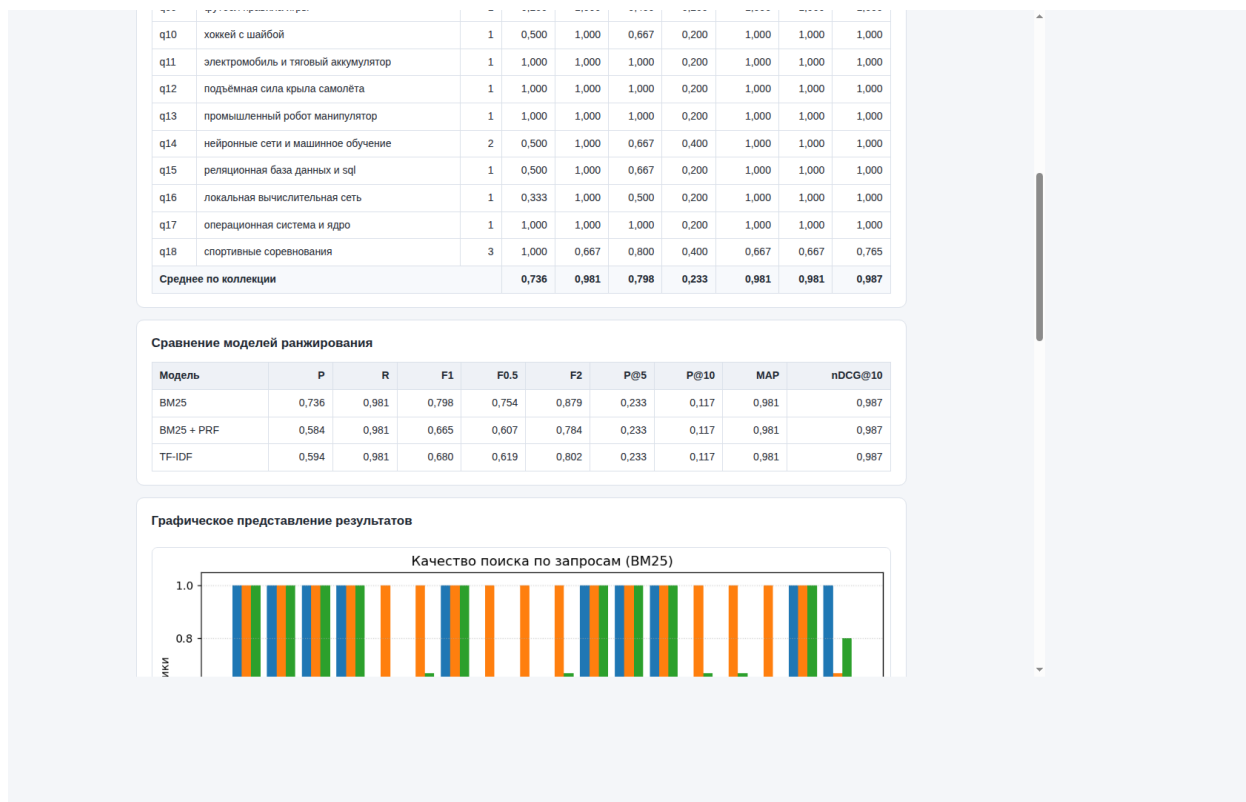


Рисунок 2.5 – Страница оценки качества поиска.

Развёртывание выполняется командой `docker compose up --build` из каталога программной части. При старте контейнер бэкенда дожидается готовности PostgreSQL, применяет миграции и, если база пуста, индексирует тестовую коллекцию. Интерфейс доступен по адресу `http://<host>:8080`.

2.4 Структура базы данных системы

Схема содержит восемь таблиц, перечисленных в таблице 2.1. Диаграмма сущностей приведена на рисунке 2.6.

Таблица	Назначение
documents	Документы коллекции: заголовок, текст, саммари, путь к файлу, длина в леммах, дата добавления
terms	Словарь: нормальная форма термина и документная частота
postings	Инвертированный индекс: частота термина в документе
search_logs	Журнал запросов: исходный текст, нормализованный образ, число результатов, время
eval_queries	Эталонные запросы для оценки качества
eval_qrels	Разметка релевантности (qrels) по имени файла документа
eval_runs	Сохранённые сводки прогонов оценки
schema_migrations	Учёт применённых SQL-миграций

Таблица 2.1 – Таблицы базы данных и их назначение.

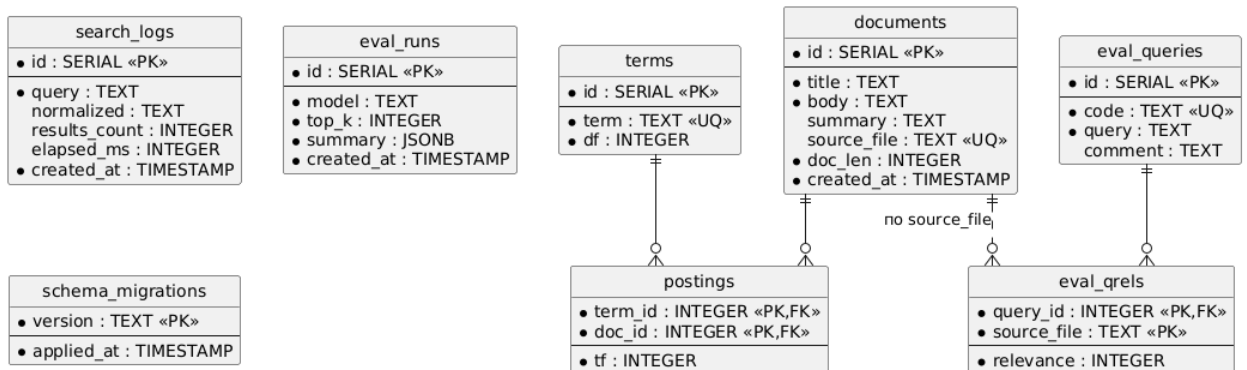


Рисунок 2.6 – Схема базы данных системы.

Связь эталонной разметки с документами выполняется по полю `source_file`, а не по числовому идентификатору: при пересоздании базы идентификаторы могут измениться, имена файлов корпуса — нет.

2.5 Основные алгоритмы реализации компонентов

Предобработка. Функция `normalize` возвращает список значимых лемм (листинг 2.1). Позиции токенов сохраняются, чтобы сниппет мог подсветить исходные словоформы, а не только нормальные формы.

Листинг 2.1 – Конвейер предобработки русскоязычного текста.

```

1 def normalize(text: str) -> list[str]:
2     return [t.lemma for t in tokenize(text) if
3             is_meaningful(t.lemma)]
4
5 def is_meaningful(lemma: str) -> bool:
6     return len(lemma) >= MIN_TOKEN_LEN and lemma not in
7         STOPWORDS

```

Индексирование. При добавлении документа частоты лемм записываются в `postings`, словарь пополняется новыми терминами, затем пересчитываются документные частоты (рисунок 2.7). Повторная загрузка файла с тем же `source_file` обновляет документ, а не создаёт дубликат.

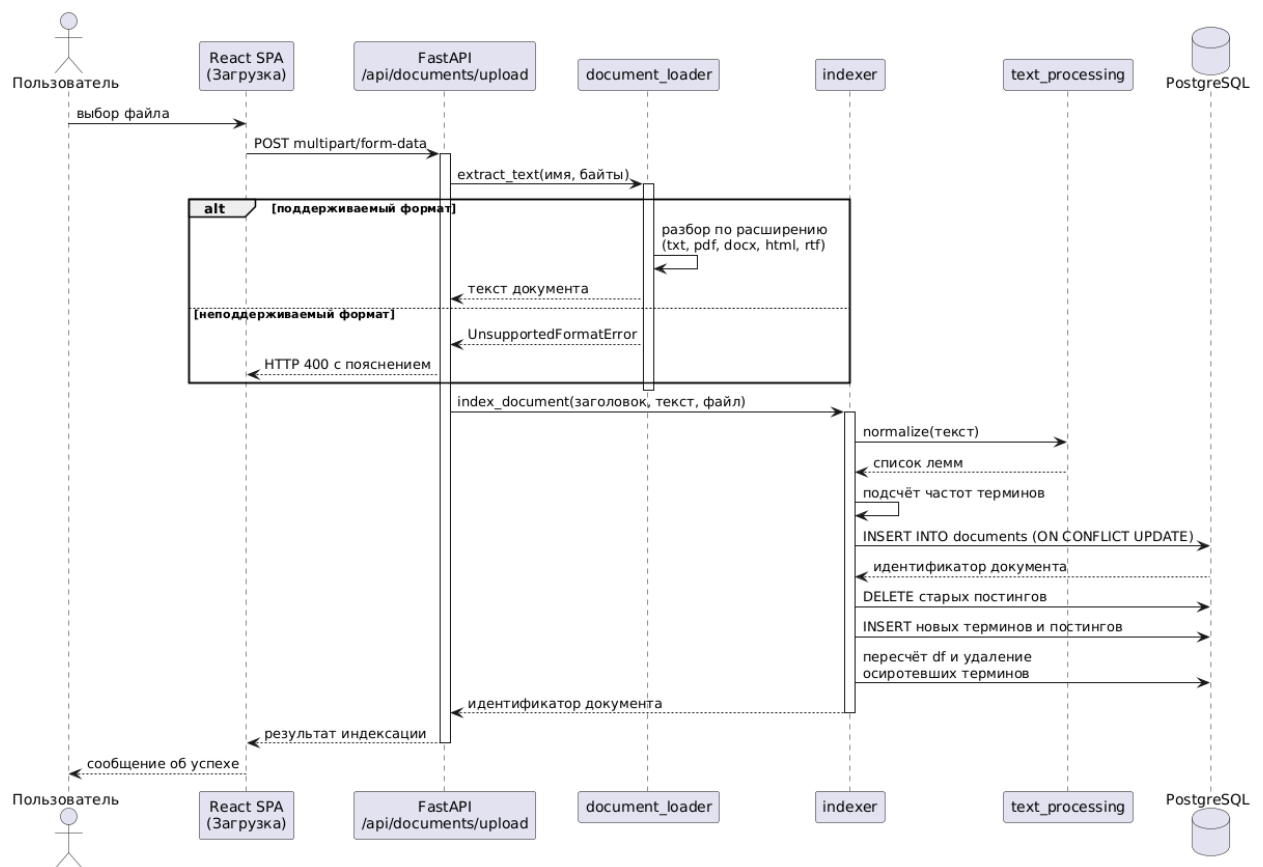


Рисунок 2.7 – Диаграмма последовательности индексирования документа.

Поддерживаемые форматы загрузки перечислены в таблице 2.2.

Расширение	Механизм извлечения текста
.txt, .md, .csv, .log	декодирование UTF-8, CP1251, KOI8-R, Latin-1
.pdf	библиотека rpypdf
.docx	библиотека python-docx
.html, .htm	удаление тегов, script- и style-блоков
.rtf	удаление управляющих слов RTF

Таблица 2.2 – Поддерживаемые форматы загружаемых документов.

Веса и BM25. Вес без обратной связи и расчёт RSV на стороне СУБД приведены в листингах 2.2 и 2.3.

Листинг 2.2 – Сглаженный вес Робертсона – Спарк Джонс.

```

1 def rsj_weight(n_docs: float, df: int) -> float:
2     return math.log(1.0 + (n_docs - df + 0.5) / (df + 0.5))

```

Листинг 2.3 – Расчёт RSV по BM25 в PostgreSQL.

```

1 SELECT p.doc_id,
2        SUM(
3            qt.w * (p.tf * (k1 + 1.0))
4            / (p.tf + k1 * (1.0 - b + b * d.doc_len /
5                stats.avgdl))
6        ) AS score
7 FROM postings p
8 JOIN qt ON qt.term_id = p.term_id
9 JOIN documents d ON d.id = p.doc_id
10 CROSS JOIN stats
11 GROUP BY p.doc_id
12 ORDER BY score DESC

```

Интеллектуальный интерфейс. По варианту 5 основным реализуемым элементом является интерфейс с пользователем. Его функции:

- автодополнение по префиксу из словаря коллекции (запрос /api/suggest);
- исправление опечаток расстоянием Дамерау – Левенштейна не более двух операций;
- расширение запроса синонимами и аббревиатурами из тезауруса («ИИ» → «искусственный интеллект»);
- сниппет с HTML-подсветкой найденных слов и отдельный список словоформ запроса, встретившихся в документе;
- фильтр выдачи по дате добавления документа;
- встроенная справка на отдельной странице.

Последовательность работы подсказок и исправления опечаток показана на рисунке 2.8.

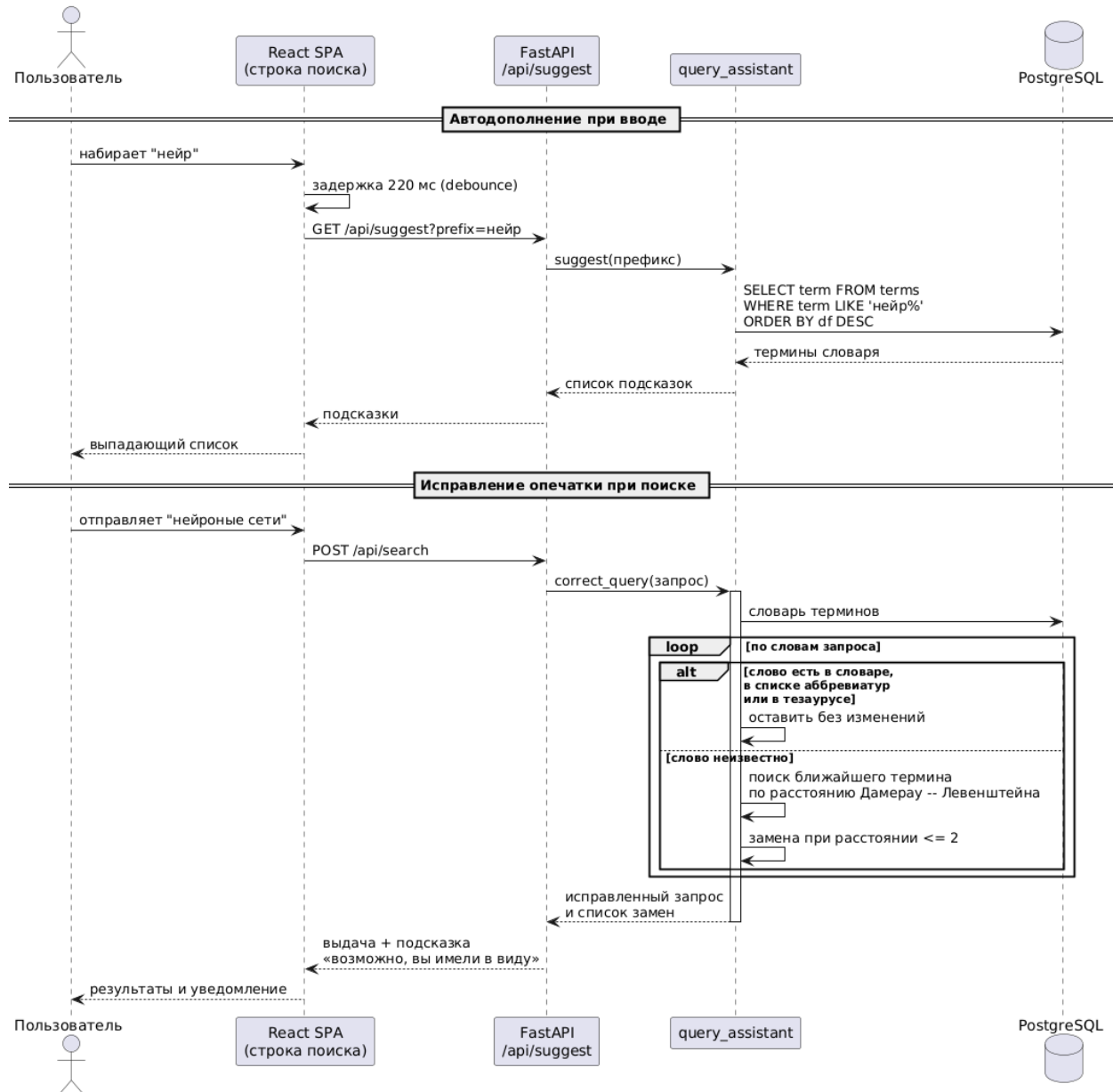


Рисунок 2.8 – Диаграмма последовательности интеллектуального интерфейса запроса.

Листинг 2.4 – Расстояние Дамерау – Левенштейна.

```

1 current[j] = min(
2     current[j - 1] + 1,          # insert
3     previous[j] + 1,             # delete
4     previous[j - 1] + cost,      # replace
5 )
6 if i > 1 and j > 1 and a[i - 1] == b[j - 2] and a[i - 2] ==
7     b[j - 1]:
8     current[j] = min(current[j], before_previous[j - 2] + cost)
  
```

Страница поиска отображает исправленный запрос («Возможно, вы имели в виду...»), добавленные синонимы, веса терминов и для каждого

документа — активную ссылку, сниппет и список найденных слов. Это закрывает требование задания о составе поисковой выдачи.

2.6 Информация о тестовой коллекции документов

Тестовая коллекция состоит из 18 русскоязычных текстов объёмом 300—600 слов, относящихся к пяти тематическим группам: естественные науки, история, спорт, техника, информационные технологии. Разнородность подобрана сознательно: на узкой однотемной коллекции вероятностная и векторная модели плохо различимы, потому что документы делят большую часть словаря. Перечень документов приведён в таблице 2.3.

ID	Файл	Тематика
1	aviaciya	гражданская авиация, подъёмная сила крыла
2	bazy_dannyh	реляционные базы данных и SQL
3	chernye_dyry	чёрные дыры, гравитация, гравитационные волны
4	drevniy_rim	Древний Рим и римское право
5	elektromobili	электромобили и тяговые аккумуляторы
6	futbol	футбол: правила и история
7	genetika_dnk	генетика, ДНК, редактирование генома
8	hockey	хоккей с шайбой
9	iskusstvennyy_intellekt	искусственный интеллект и его применение
10	kosmos_issledovaniya	исследование космоса, спутники, Марс
11	lokalnye_seti	локальные вычислительные сети
12	neyronnye_seti	нейронные сети и машинное обучение
13	olimpiyskie_igry	Олимпийские игры
14	operacionnye_sistemy	операционные системы и ядро
15	robototekhnika	промышленная робототехника
16	vakciny_immunitet	вакцинация и иммунная система
17	velikoe_knyazhestvo	Великое княжество Литовское
18	vtoraya_mirovaya	Вторая мировая война

Таблица 2.3 – Тестовая коллекция документов.

Документы загружаются автоматически при пустой базе из каталога `backend/app/corpus/` в алфавитном порядке имён файлов: идентификатор в таблице 2.3 совпадает с полем `id` в базе. Эталонная разметка хранится в файле `qrels.json`: 18 запросов с указанием релевантных файлов. После индексации разметка привязывается к числовым идентификаторам через `source_file`.

2.7 Результаты тестирования системы

Тестирование проводилось автоматически сценарием `backend/scenarios.py` и вручную через веб-интерфейс. Проверялись следующие сценарии:

- а) предобработка: лемматизация словоформ, отсев стоп-слов, сохранение латиницы;
- б) инициализация: 18 документов, словарь из 2091 термина, 3313 постингов, отсутствие дубликатов при повторном старте;
- в) вероятностные веса: редкий термин получает больший вес, чем частый (2,539 против 0,204 на контрольной паре);
- г) поиск: однословный запрос, многословный запрос, запрос из слов, отсутствующих в коллекции;
- д) интерфейс: исправление опечатки «нейроны» → «нейронный», расширение аббревиатуры «ИИ», автодополнение по префиксу, подсветка сниппета, список найденных словоформ;
- е) загрузка: извлечение текста из HTML, отказ неподдерживаемого формата с понятным сообщением;
- ж) индексирование и удаление: новый документ становится доступен поиску и исчезает из индекса после удаления;
- з) оценка качества: расчёт таблиц и графиков по эталонным запросам.

Все 20 автоматических проверок завершились успешно. Средняя длина документа составила 232,9 леммы.

2.8 Результаты оценки по метрикам

Оценка выполнялась на 18 эталонных запросах. Для каждого запроса система возвращала выдачу с отсечением хвоста, после чего считались метрики (1.5), $P@k$, MAP и nDCG@10. Последовательность расчёта показана на рисунке 2.9. Результаты по запросам для модели BM25 без обратной связи приведены в таблице 2.4.

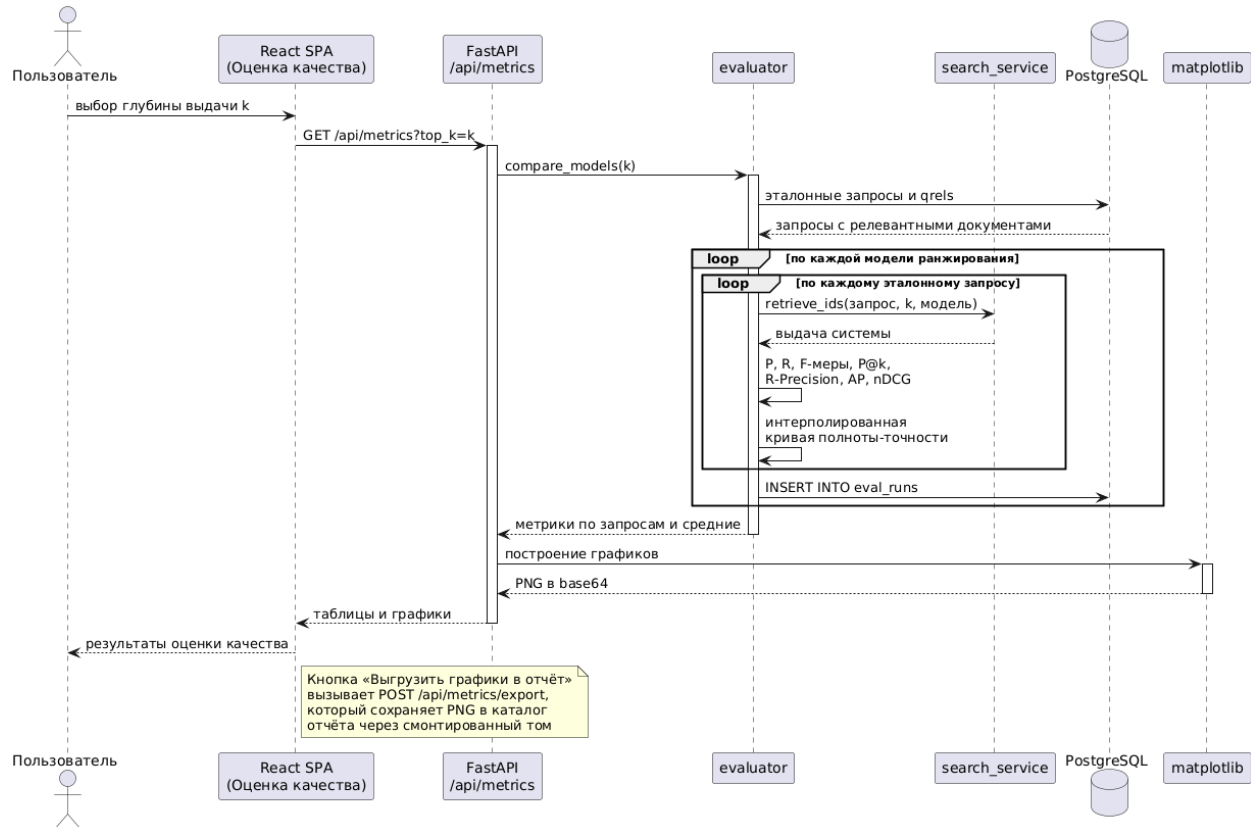


Рисунок 2.9 – Диаграмма последовательности оценки качества поиска.

Запрос	P	R	F1	P@5	AP
исследование космоса и планет	1,000	1,000	1,000	0,400	1,000
чёрные дыры и гравитационные волны	1,000	1,000	1,000	0,200	1,000
вакцинация и иммунитет человека	1,000	1,000	1,000	0,200	1,000
наследственность гены и днк	1,000	1,000	1,000	0,200	1,000
вторая мировая война	0,333	1,000	0,500	0,200	1,000
древний рим и римское право	0,500	1,000	0,667	0,200	1,000
великое княжество литовское	1,000	1,000	1,000	0,200	1,000
олимпийские игры	0,333	1,000	0,500	0,200	1,000
футбол правила игры	0,250	1,000	0,400	0,200	1,000
хоккей с шайбой	0,500	1,000	0,667	0,200	1,000
электромобиль и тяговый аккумулятор	1,000	1,000	1,000	0,200	1,000
подъёмная сила крыла самолёта	1,000	1,000	1,000	0,200	1,000
промышленный робот манипулятор	1,000	1,000	1,000	0,200	1,000
нейронные сети и машинное обучение	0,500	1,000	0,667	0,400	1,000
реляционная база данных и sql	0,500	1,000	0,667	0,200	1,000
локальная вычислительная сеть	0,333	1,000	0,500	0,200	1,000
операционная система и ядро	1,000	1,000	1,000	0,200	1,000
спортивные соревнования	1,000	0,667	0,800	0,400	0,667
Среднее	0,736	0,981	0,798	0,233	0,981

Таблица 2.4 – Метрики качества BM25 по тестовым запросам.

Сравнение трёх моделей ранжирования приведено в таблице 2.5.

Модель	P	R	F1	P@5	MAP	nDCG@10
BM25	0,736	0,981	0,798	0,233	0,981	0,987
BM25 + PRF	0,584	0,981	0,665	0,233	0,981	0,987
TF-IDF	0,594	0,981	0,680	0,233	0,981	0,987

Таблица 2.5 – Сравнение моделей ранжирования на тестовой коллекции.

Графическое представление — столбчатая диаграмма точности, полноты и F1 по запросам (рисунок 2.10), 11-точечная кривая полноты-точности (рисунок 2.11) и сравнение моделей (рисунок 2.12).

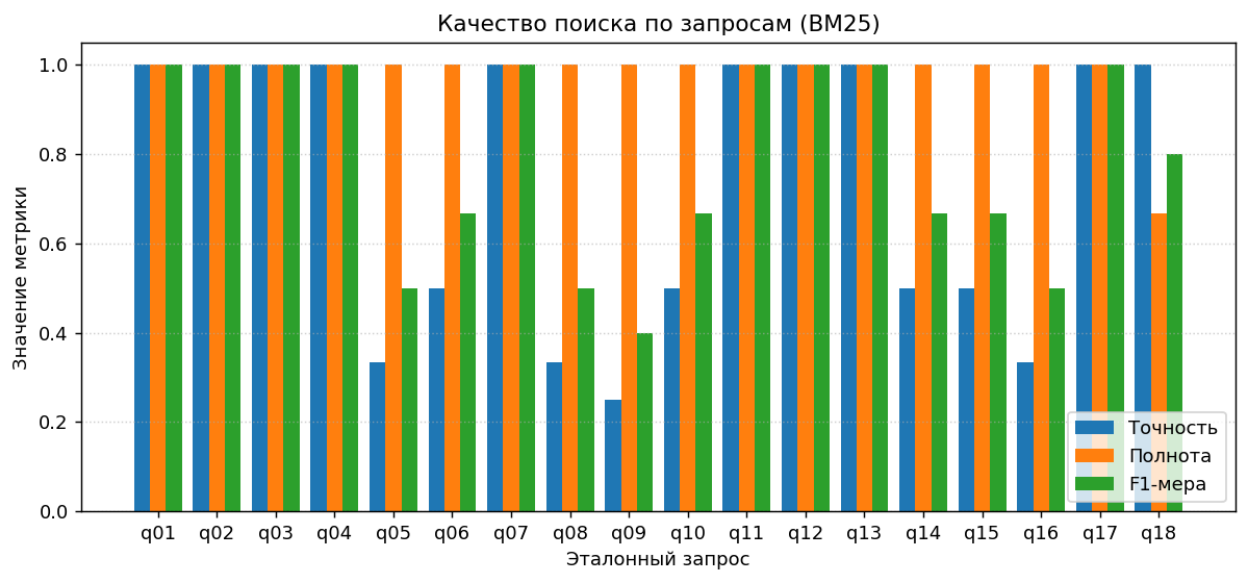


Рисунок 2.10 – Точность, полнота и F1-мера по эталонным запросам (BM25).

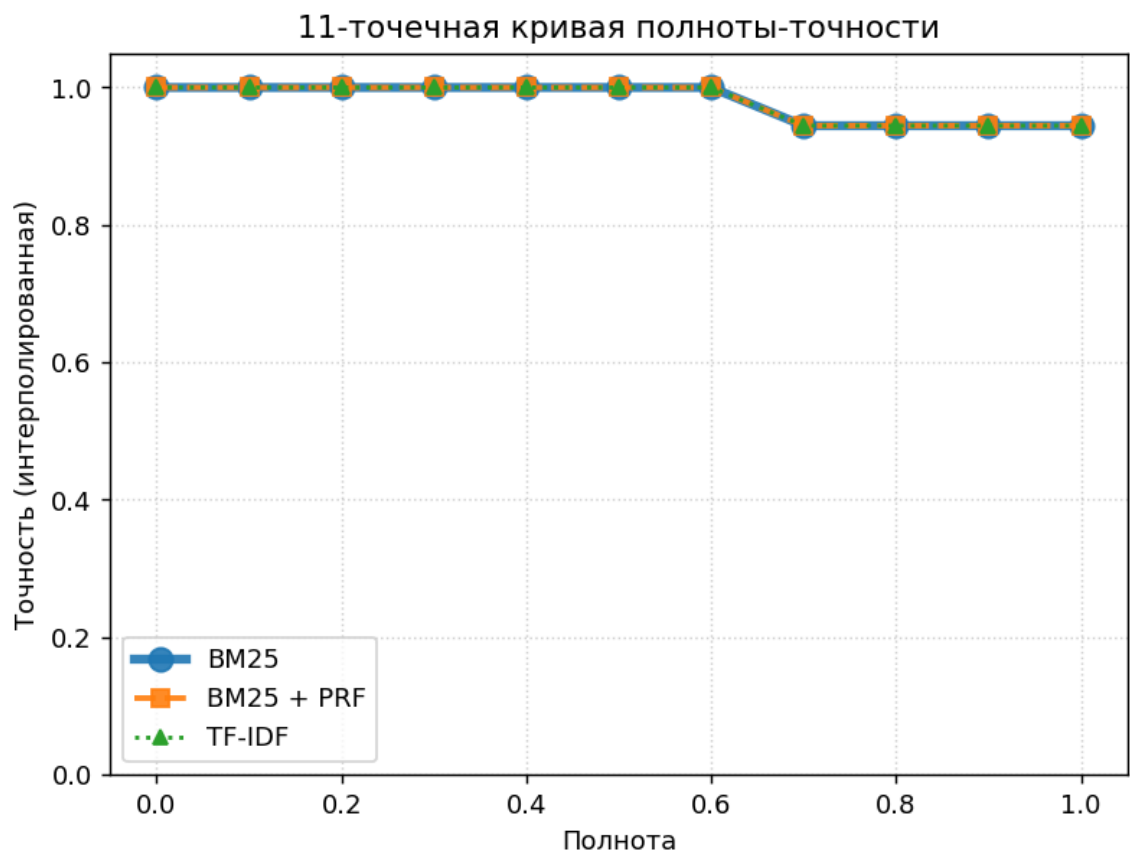


Рисунок 2.11 – 11-точечная интерполированная кривая полноты-точности.

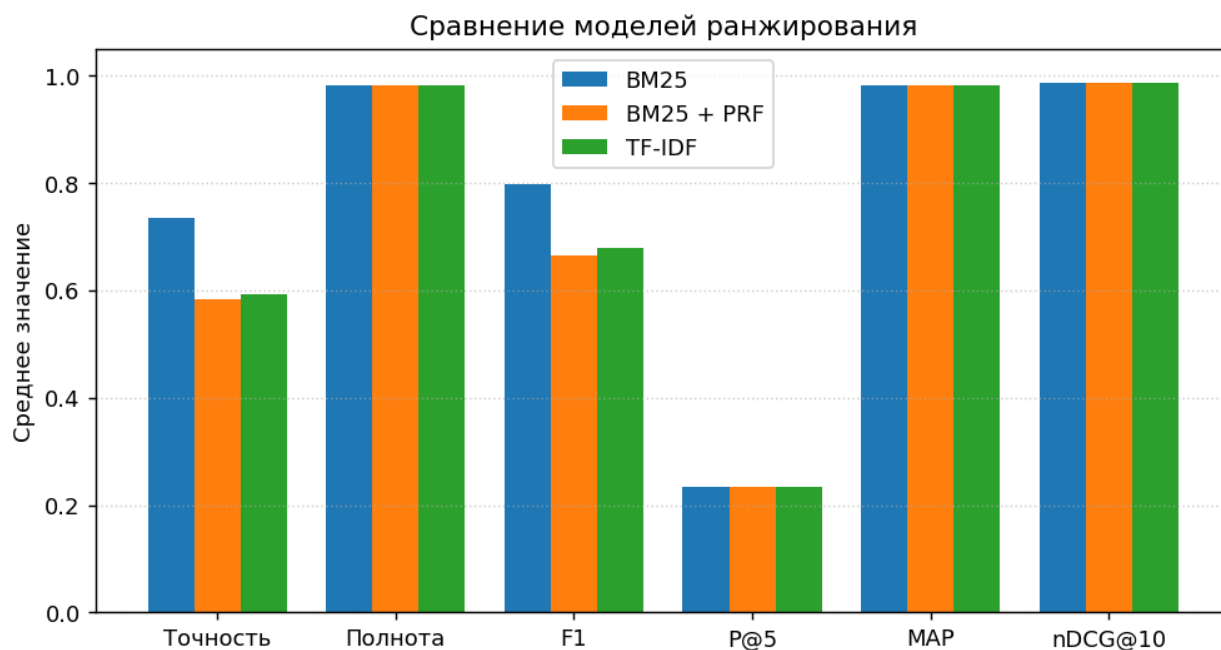


Рисунок 2.12 – Сравнение BM25, BM25 с обратной связью и TF-IDF.

2.9 Анализ результатов и предложения по улучшению

Анализ полученных данных показывает следующее.

- Полнота близка к единице (0,981): на коллекции из 18 документов релевантные тексты почти всегда попадают в выдачу. Исключение — широкий запрос «спортивные соревнования», для которого не найден один из трёх эталонных документов.

- Точность ограничивается общими словами запроса. Для «футбол правила игры» слово «игра» встречается в нескольких спортивных текстах, и отсечение хвоста оставляет четыре документа при одном эталонном ($P = 0,250$). Специфичные запросы с редкими терминами («чёрные дыры», «тяговый аккумулятор») дают $P = 1$.

- MAP и nDCG@10 почти равны единице: когда релевантный документ найден, он стоит на первой позиции. Качество ранжирования на данной коллекции выше, чем качество отсечения нерелевантного хвоста.

- $P@5$ и $P@10$ занижены по построению: в знаменателе всегда стоит k , тогда как релевантных документов обычно один-два, а выдача после отсечения короче пяти позиций.

- BM25 превосходит TF-IDF по точности (0,736 против 0,594) и F1-мере (0,798 против 0,680). Насыщение частоты и поправка на длину уменьшают вклад случайных совпадений.

- Псевдорелевантная обратная связь на этой коллекции вредна: точность падает до 0,584. При $R = 5$ в псевдорелевантное множество попадают документы соседних тем, и пересчёт весов усиливает общие слова.

Предложения по улучшению работы системы:

- а) расширить коллекцию до сотен документов, чтобы полнота и $P@k$ стали информативнее, а обратная связь — полезнее;
- б) учитывать биграммы и устойчивые словосочетания («вторая мировая война», «локальная вычислительная сеть»);
- в) подбирать порог отсечения и параметр R по валидационным запросам, а не фиксировать их заранее;
- г) заменить ручной тезаурус векторными эмбедингами или готовым словарём синонимов;
- д) добавить оценку по кликам пользователя как истинную обратную связь вместо псевдорелевантной;
- е) вынести расчёт метрик в регулярный прогон при изменении корпуса.

2.10 Вывод по разделу

Реализована информационно-поисковая система с вероятностной моделью Okapi BM25, инвертированным индексом в PostgreSQL и интеллектуальным веб-интерфейсом. Система поддерживает загрузку документов, поиск по естественно-языковому запросу с подсветкой найденных слов, автодополнение, исправление опечаток и программную оценку качества. На тестовой коллекции средняя F1-мера BM25 составила 0,798 при полноте 0,981, что выше результата базовой векторной модели.

Заключение

В рамках работы разработана информационно-поисковая система, реализующая интерфейс с пользователем на основе вероятностной модели поиска Okapi BM25. Система разворачивается в локальной вычислительной сети средствами Docker Compose и предназначена для работы с документами на русском языке, что соответствует варианту 5 задания.

В ходе выполнения работы решены следующие задачи:

- спроектирована и реализована трёхуровневая архитектура: веб-интерфейс на React, серверная часть на Python 3.11 (FastAPI) и уровень данных на PostgreSQL 16;

- спроектирована база данных из восьми таблиц: документы, словарь, инвертированный индекс, журналы поиска, эталонные запросы, разметка релевантности, прогоны оценки и учёт миграций;

- реализованы алгоритмы предобработки русского текста (токенизация, лемматизация, фильтрация стоп-слов), построения инвертированного индекса, вычисления весов Робертсона – Спарк Джонс и ранжирования по BM25;

- реализован интеллектуальный интерфейс: автодополнение, исправление опечаток, расширение запроса синонимами, сниппеты с подсветкой и список слов запроса, найденных в документе, встроенная справка;

- реализована загрузка документов форматов TXT, MD, PDF, DOCX, HTML, RTF, CSV и LOG;

- реализована программная оценка качества: точность, полнота, F-меры, P@k, MAP, nDCG и 11-точечная кривая полноты-точности, с выводом таблиц и графиков;

- обеспечено воспроизводимое развёртывание Docker Compose с автоматическим применением миграций и загрузкой корпуса.

Тестирование подтвердило работоспособность предобработки, индексирования, поиска, интеллектуального интерфейса, загрузки файлов и оценки качества. На коллекции из 18 разнородных русскоязычных документов и 18 эталонных запросов модель BM25 показала среднюю точность 0,736, полноту 0,981 и F1-меру 0,798, превзойдя базовую векторную модель TF-IDF.

Перспективными направлениями улучшения являются расширение корпуса, учёт словосочетаний, адаптивный порог отсека выдачи, замена ручного тезауруса на векторные представления и использование подлинной обратной связи пользователей.

Поставленная цель достигнута: система работоспособна, соответствует вероятностной стратегии поиска и может использоваться как основа для дальнейших исследований в области информационного поиска.

Список использованных источников

- [1] Маннинг К., Рагхаван П., Шютце Х. Введение в информационный поиск. — М. : Вильямс, 2011. — 528 с.
- [2] Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. — 2009. — Vol. 3, No. 4. — P. 333–389.
- [3] Robertson S. E., Sparck Jones K. Relevance weighting of search terms // Journal of the American Society for Information Science. — 1976. — Vol. 27, No. 3. — P. 129–146.
- [4] Российский семинар по оценке методов информационного поиска (ROMIP). Метрики качества поиска. — 2004.
- [5] Korobov M. Morphological Analyzer and Generator for Russian and Ukrainian Languages // Analysis of Images, Social Networks and Texts. — 2015. — P. 320–332.
- [6] FastAPI documentation [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/>.
- [7] PostgreSQL 16 Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/16/>.
- [8] React documentation [Электронный ресурс]. — Режим доступа: <https://react.dev/>.